(Optional)Coding Challenge: Advanced Object Oriented Programming

Estimated time needed: 20 minutes

In this self-guided lab, you can complete exercises that align with what you learned in this module. Based on what you learned in this module learning, you apply what you've learned to create the apps. Then, verify if your code is correct using the solutions provided.

Prerequisites (optional)

You must have completed all the labs in Modules 1 and 2.

Clone the challenges repo

1. Run the following command to clone the repository with the challenges and solution.

```
git clone https://github.com/ibm-developer-skills-network/flgsb-oop-additional-practice.git
```

2. Change to `OOPs/Module 2` directory.

```
cd flgsb-oop-additional-practice/OOPs/Module\ 2
```

3. View the directories by running the `ls` command. Each directory has one challenge in it.

```
ls
```

Abstract Classes and Interfaces Challenge

Introduction

Welcome to the Abstract Classes and Interfaces Challenge! This exercise will help you understand and implement key concepts of object-oriented programming in Java:

- Abstract methods and abstract classes
- Interfaces and their implementation
- Polymorphism with abstract types
- Multiple interface inheritance

In this challenge, you'll create a system for various musical instruments that demonstrates how to use abstract classes and interfaces to model real-world relationships and behaviors. You'll see how these concepts enable code reuse, polymorphism, and proper code organization.

What You'll Learn

By completing this challenge, you'll gain hands-on experience with:

- Creating and implementing abstract methods
- Defining abstract classes and extending them
- Creating interfaces and implementing them
- Using polymorphism to work with different objects through common interfaces
- Understanding the difference between abstract classes and interfaces
- Implementing multiple interfaces to achieve a form of multiple inheritance

Solve the challenge

1. In the terminal window, change to `Designing interfaces and abstract classes`.

   ```
   cd Designing\ interfaces\ and\ abstract\ classes
   ```

2. Select the following button to open the Java coding challenge.

Open **MusicShop.java** in IDE

3. Make changes in the file based on the specifications.

4. Compile the Java file.

5. Set the CLASSPATH.

6. Run the class `MusicShop` to verify the class is created based on the instructions.

Click here for the solution

Implementing Polymorphism

This challenge will help you practice and understand the concept of polymorphism in Java, one of the four fundamental object-oriented programming principles. Polymorphism allows objects of different classes to be treated as objects of a common superclass, with methods being called based on the actual object type rather than the reference type.

In this challenge, you'll build a Shape Hierarchy System by creating a Shape superclass and various shape subclasses that inherit from it. This lab demonstrates how polymorphism enables flexible and extensible code design.

What You'll Learn

- How to create a superclass with common methods
- How to create subclasses that inherit from the superclass
- How to override methods in subclasses
- How to use polymorphism to create instances of different shapes
- How to store subclass objects in an array of the superclass type

The Challenge

You'll create several Java classes:

- Shape.java: The superclass with common methods
- Circle.java, Rectangle.java, Triangle.java - Subclasses that extend Shape
- ShapeDemo.java: A client program that demonstrates polymorphism

Solve the challenge

1. In the terminal, change to `Create your own class`.

   ```
   cd Implementing\ Polymorphism
   ```

2. Select the following button to open `Shape.java`

Open **Shape.java** in IDE

3. Make changes in the file based on the specifications.
4. Compile the Java file.
5. Set the CLASSPATH.
6. Select the following button to open `Circle.java`

Open **Circle.java** in IDE

7. Make changes in the file based on the specifications.
8. Compile the Java file.
9. Select the following button to open `Rectangle.java`

Open **Rectangle.java** in IDE

10. Make changes in the file based on the specifications.
11. Compile the Java file.
12. Select the following button to open `Triangle.java`

Open **Triangle.java** in IDE

13. Make changes in the file based on the specifications.
14. Compile the Java file.
15. Select the following button to open `ShapeDemo.java`

Open **ShapeDemo.java** in IDE

16. Make changes in the file based on the specifications.
17. Compile the Java file.
18. Run the class `ShapeDemo` to verify that the class is created based on the instructions.

Click here for the solution.

Conclusion

After you complete these labs, you should be able to demonstrate your skills with inheritance and polymorphism object oriented programming.

Author(s)

Lavanya

© IBM Corporation. All rights reserved.